Decision on Type Validation and Typed Loading

Giuseppe Rudi

July 16, 2026

During the design of the Data Quality Assessment and Data Cleaning workflow, two possible strategies were considered.

The first strategy was to load all re-engineered CSV files into a staging PostgreSQL schema where every column was stored as text. This solution would allow the DQA to explicitly test whether each value can be converted into the expected type, such as date, boolean, integer, or floating-point number.

However, this strategy is more suitable for uncontrolled CSV sources, where all values are initially textual and the project must infer and validate the correct data type of each column.

This is not the main situation of the present project.

In this project, the main data source is the FastF1 Python API. FastF1 does not provide uncontrolled textual CSV files as the primary input. Instead, it returns structured Pandas DataFrames, where columns already have an expected computational type or a structured representation.

During the re-engineering phase, the project selects the relevant attributes, adds structural identifiers, converts time-related attributes into integer milliseconds, and exports coherent reconciled CSV files.

For this reason, the project adopts a typed loading strategy.

The re-engineered CSV files are loaded into PostgreSQL tables with explicit data types. This choice is not used as a replacement for data quality assessment. Rather, it reflects the fact that basic type interpretation has already been handled upstream by the API and by the controlled Python re-engineering pipeline.

Therefore, the post-load DQA does not focus on pure type parseability checks such as:

- whether a timestamp column can be parsed as a timestamp;
- whether a boolean column can be parsed as a boolean;
- whether a numerical column can be parsed as a number.

These checks would be more relevant if the project started from uncontrolled textual files. In this project, they would add complexity without providing significant analytical value.

Instead, the DQA focuses on the quality problems that are still possible and relevant after typed loading.

A value may have the correct technical type but still be invalid, inconsistent, suspicious, or analytically unusable.

For example:

- a speed value can be numerical but negative or unrealistic;
- a lap time can be numerical but equal to zero or inconsistent with sector times;
- two dates can be valid timestamps but logically inconsistent with each other;
- a tyre compound can be textual but outside the expected Formula 1 compound domain;
- a row can have valid types but missing identifiers;
- two rows can be individually valid but duplicated according to a natural key;

- a foreign key can refer to a non-existing parent row;
- a lap can be valid as data but not suitable for clean performance analysis.

For this reason, the project distinguishes between two levels of validity:

- **type-level validity**, which concerns whether a value can be represented with the expected technical type;
- **domain-level and semantic validity**, which concerns whether a value is meaningful, coherent, and usable in the Formula 1 analytical domain.

The first level is mostly handled by the source API, the re-engineering script, and the typed loading process. The second level is the main focus of the Data Quality Assessment and Data Cleaning phases.

Examples of post-load DQA checks are:

- missing required identifiers;
- duplicated primary or natural keys;
- invalid categorical domains;
- impossible or unrealistic numerical values;
- inconsistencies between related attributes;
- mismatches between related tables;
- suspicious or unreliable laps;
- referential integrity problems.

This decision keeps the DQA and Data Cleaning phases focused on the problems that are most relevant for the construction of the Data Warehouse.

In particular, the project does not attempt to duplicate basic type inference already provided by the API and by the re-engineering pipeline. Instead, it evaluates whether the typed and loaded values are complete, consistent, realistic, referentially correct, and analytically useful.

Canonical Typed Reconciled Schema

The typed load creates exactly ten normalized reconciled tables. The following compact schema records the PostgreSQL type families used by the loader; nullable analytical attributes remain nullable unless a later constraint is explicitly introduced.

Table	Identifier and links	Typed attributes
`season`	`season_year BIGINT`	season dates `TIMESTAMP`; event count `BIGINT`; champion descriptors `TEXT`.
`circuit`	`circuit_id TEXT`	name, country, location, provenance and circuit/sector categories `TEXT`.
`grand_prix`	`grand_prix_id BIGINT`; season and circuit links	round `BIGINT`; event date `TIMESTAMP`; API support `BOOLEAN`; descriptors `TEXT`.
`session`	`session_id BIGINT`; Grand Prix link	type/name `TEXT`; date `TIMESTAMP`.

Table	Identifier and links	Typed attributes
driver	driver_id TEXT	permanent number DOUBLE PRECISION; date of birth TIMESTAMP; abbreviation, names, nationality, URL TEXT.
team	team_id TEXT	team name, nationality, URL TEXT.
result	result_id, session_id BIGINT; driver/team links TEXT	positions, points and laps numeric; Q1/Q2/Q3 and time BIGINT; status/classification TEXT.
lap	lap_id, session_id BIGINT; driver/team links TEXT	all time measures, including lap_start_time_ms, are BIGINT; speeds/numeric counters DOUBLE PRECISION; flags BOOLEAN; descriptors TEXT; start date TIMESTAMP.
weather	weather_id, session_id, time_ms BIGINT	temperatures, humidity, pressure, wind values DOUBLE PRECISION; rainfall BOOLEAN.
track_status	track_status_id, session_id, time_ms BIGINT	status and message TEXT.

Result-position nullability. grid_position is a nullable numeric attribute: a present value must be non-negative; missing values are expected for Qualifying and are preserved as null. A missing or invalid value in Race is handled as a race-context quality issue. By contrast, position is required for an analytically usable result row.

Canonical naming. Columns are normalized to snake case. The canonical names include speed_st, abbreviation, nationality, session_id, and lap_start_time_ms. Source identifiers lap_id and result_id belong to the reconciled and audit layers; they are not exported as analytical fact identifiers.

Separation from the Warehouse

Typed loading does not denormalize the reconciled schema. Denormalization and surrogate warehouse keys are introduced only by the ETL: the physical facts use lap_performance_key and session_result_key, while the circuit attributes are incorporated into dim_grand_prix. This prevents warehouse decisions from being confused with source typing and reconciled relational constraints.